

DOCUMENTO DE ESPECIFICACIÓN LÓGICA Y MATEMÁTICA

Arquitectura de Algoritmos, Teoremas de Periodo y Estructura del Sistema PRNG

Asignatura: Simulación Digital	Módulo: Generadores de Números Pseudoaleatorios (PRNG)
Autores: Adrian Angulo M. & Samuel Molina R.	Tecnología: Python 3 / FastAPI / OpenPyXL
Alcance: Lógica Matemática, Validaciones y Algoritmos	Fecha de Revisión: Agosto 2026

1. Arquitectura del Código y Ubicación de la Lógica

Toda la lógica de negocio, validaciones formales de teoría de números, generación de secuencias y cálculos estocásticos se encuentra estrictamente modularizada en la capa de backend dentro del paquete core/ y el despachador app.py. A continuación se detalla la responsabilidad de cada archivo:

Archivo / Módulo	Responsabilidad y Funciones Principales	Lógica que Implementa
core/validators.py	• validate_gclm() • validate_gcm() • validate_cuadrados_medios() • validate_productos_medios() • validate_bbs() • Utilidades: is_prime(), gcd(), find_primitive_roots(), is_power_of_two()	Verificación matemática de dominios, Teorema de Hull-Dobell, raíces primitivas mod m, periodo maximal de Lehmer, primos de Blum ($p \equiv 3 \pmod{4}$) y heurísticas de von Neumann.
core/generators.py	• generate_gclm() • generate_gcm() • generate_cuadrados_medios() • generate_productos_medios() • generate_bbs() • calculate_stats()	Máquinas de estado iterativas. Generación de trazas paso a paso, detección de ciclos por historial de estados, cálculo de media, varianza, retardo Lag-1 (R_i vs R_{i+1}) y autocorrelación.
core/recommenders.py	• generate_gclm_optimal() • generate_gcm_optimal() • generate_cuadrados_medios_optimal() • generate_productos_medios_optimal() • generate_bbs_optimal()	Síntesis constructiva de parámetros óptimos que garantizan periodo maximal según los teoremas algebraicos sin intervención manual.
core/excel_exporter.py	• export_prng_to_excel() • _build_sheet1() • _build_sheet2()	Construcción del libro .xlsx profesional en memoria con OpenPyXL: tabla de parámetros, secuencia numérica con 6 decimales y gráfico de dispersión nativo.
app.py	• /api/generate • /api/validate • /api/recommend • /api/download/{filename}	Controlador REST API (FastAPI) que desacopla la comunicación frontend-backend y gestiona descargas nativas con FileResponse.

2. Análisis Detallado de los 5 Métodos Pseudoaleatorios

A continuación se expone la formulación matemática exacta, restricciones de dominio, teoremas de periodo, características operativas y modo de fallo de cada uno de los algoritmos implementados.

2.1 Generador Congruencial Lineal Mixto (GCLM)

Formulación Recursiva:

$$X_{i+1} = (a \cdot X_i + c) \bmod m \quad | \quad R_i = X_i / m \in [0, 1)$$

Parámetros y Dominio:

- **X₀** (Semilla Inicial): Entero no negativo en el rango $0 \leq X_0 < m$.
- **a** (Multiplicador): Entero constante $1 < a < m$.
- **c** (Incremento / Constante Aditiva): Entero constante $1 \leq c < m$.
- **m** (Módulo): Entero positivo $m > \max(X_0, a, c)$.

Teorema Fundamental de Hull-Dobell (Periodo Completo $N = m$):

El generador tiene periodo completo $N = m$ (genera todos los números entre 0 y $m-1$ sin repetir ninguno antes) **si y sólo si** se cumplen simultáneamente las siguientes 3 condiciones algebraicas:

Condición 1:	$\text{mcd}(c, m) = 1$ (c y m deben ser estrictamente coprimos).
Condición 2:	Todo factor primo p de m debe dividir exactamente a (a - 1), es decir, $(a - 1) \bmod p = 0$.
Condición 3:	Si 4 divide a m ($m \bmod 4 = 0$), entonces 4 debe dividir a (a - 1) ($(a - 1) \bmod 4 = 0$).

Características: Altísima velocidad de cómputo. Si Hull-Dobell se cumple, la elección de la semilla X_0 no afecta la longitud del periodo completo $N = m$.

2.2 Generador Congruencial Multiplicativo (GCM / Lehmer)

Formulación Recursiva:

$$X_{i+1} = (a \cdot X_i) \bmod m \quad | \quad R_i = X_i / m \in (0, 1) \quad (c = 0)$$

Parámetros y Dominio:

- **X₀** (Semilla Inicial): Entero positivo $0 < X_0 < m$. $X_0 \neq 0$ (0 es un estado absorbente que anula la secuencia).
- **a** (Multiplicador): Entero constante $1 < a < m$.
- **m** (Módulo): Entero positivo mayor que a y X_0 .

Reglas de Periodo Maximal:

Al carecer de término aditivo ($c=0$), el GCM nunca puede alcanzar periodo $N=m$. Su periodo maximal depende de la estructura algebraica de m:

Caso A: Módulo Potencia de 2 ($m = 2^g$ con $g \geq 4$)	• Periodo maximal alcanzable: $N = m / 4 = 2^{(g-2)}$. • Restricción de semilla: X_0 debe ser impar ($\text{mcd}(X_0, m) = 1$). • Restricción del multiplicador: $a = 3 + 8k$ o $a = 5 + 8k$ (con k entero ≥ 0).
Caso B: Módulo Primo (m es un número primo)	• Periodo maximal alcanzable: $N = m - 1$. • Restricción de semilla: Cualquier $0 < X_0 < m$. • Restricción del multiplicador: a debe ser una raíz primitiva módulo m, es decir, el orden multiplicativo de a mod m debe ser exactamente $m - 1$ (para todo factor primo q de $m-1$: $a^{(m-1)/q} \bmod m \neq 1$).

2.3 Algoritmo de Cuadrados Medios (John von Neumann, 1949)

Mecánica del Algoritmo:

1. Se toma la semilla actual de D dígitos X_i (con D par, comúnmente $D=4$).
2. Se eleva al cuadrado: $Y_i = (X_i)^2$ (produce hasta $2D$ dígitos).
3. Si Y_i tiene menos de $2D$ dígitos, se rellena con ceros a la izquierda hasta longitud $2D$.
4. Se extraen los D dígitos centrales como el siguiente valor X_{i+1} .
5. El número normalizado se obtiene como: $R_i = X_{i+1} / 10^D \in [0, 1)$.

Patologías y Modos de Fallo (Heurísticas de Validación):

- **Colapso a Cero:** Si los dígitos centrales extraídos son 0000, todas las iteraciones subsiguientes se convierten en cero indefinidamente.
- **Punto Fijo Degenerativo:** Ciertas semillas generan su propio valor al cuadrado (ej. $3792^2 = 14379264 \rightarrow 3792$), deteniendo la variabilidad.
- **Terminaciones Riesgosas:** Semillas que terminan en 00, 25, 50 o 75 multiplican ceros hacia el centro en pocas iteraciones.
- **Periodos Cortos:** Carece de base matemática para garantizar periodo largo; es un método histórico no congruencial.

2.4 Algoritmo de Productos Medios

Mecánica del Algoritmo:

1. Requiere dos semillas iniciales de D dígitos: X_i y X_j (con $X_i \neq X_j$).
2. En el paso i , se multiplican los dos estados previos: $Y_i = X_{i-1} \cdot X_i$.
3. Se rellena Y_i con ceros a la izquierda a longitud $2D$.
4. Se extraen los D dígitos centrales como X_{i+1} .
5. $R_i = X_{i+1} / 10^D$.

Restricciones y Ventajas:

- **Condición de Semillas:** $\text{mcd}(X_i, X_j) = 1$ (coprimas) y longitud exacta de D dígitos sin ceros terminales.
- **Mejora frente a Cuadrados Medios:** Al involucrar dos estados en la multiplicación, reduce sustancialmente la tasa de colapso a puntos fijos y alarga el ciclo promedio.

2.5 Generador Criptográfico Blum Blum Shub (BBS, 1986)

Formulación y Base Criptográfica:

Blum Blum Shub es un generador pseudoaleatorio criptográficamente seguro (CSPRNG). Su seguridad matemática se fundamenta en la intratabilidad computacional del **Problema de Residuos Cuadráticos** y la dificultad de factorizar el entero de Blum $M = p \cdot q$.

Mecánica Iterativa:

- **Entero de Blum:** $M = p \cdot q$, donde p y q son primos grandes con $p \equiv 3 \pmod{4}$ y $q \equiv 3 \pmod{4}$.
- **Semilla Inicial:** s tal que $\text{mcd}(s, M) = 1$ y $s \notin \{0, 1\}$.
- **Estado Inicial ($i = 0$):** $X_0 = (s)^2 \bmod M$.
- **Evolución del Estado:** $X_{i+1} = (X_i)^2 \bmod M$.
- **Extracción de Bit Criptográfico:** $b_i = X_i \bmod 2$ (bit de paridad / LSB).
- **Número Pseudoaleatorio Normalizado:** $R_i = X_i / M \in (0, 1)$.

Longitud del Periodo: El ciclo en el grupo de residuos cuadráticos divide a $\lambda(\lambda(M))$, donde $\lambda(M) = \text{mcm}(p-1, q-1)$ es la función indicatriz de Carmichael.

3. Matriz Comparativa de Algoritmos PRNG

Método	Fórmula de Avance	Periodo Maximal	Seguridad / Aplicación
GCLM (Mixto)	$X_{i+1} = (aX_i + c) \bmod m$	$N = m$ (Hull-Dobell)	No criptográfico. Excelente para simulación de Monte Carlo e ingeniería.
GCM (Lehmer)	$X_{i+1} = (aX_i) \bmod m$	$N = m/4$ (2 ¹⁶) o $N = m-1$ (primo)	No criptográfico. Muy rápido computacionalmente (sin sumas).
Cuadrados Medios	$X_{i+1} = \text{Centro_D}((X_i)^2)$	Impredecible (corto)	Histórico / Didáctico. Riesgo de colapso a 0000 o bucles cortos.
Productos Medios	$X_{i+1} = \text{Centro_D}(X_{i-1} \cdot X_i)$	Impredecible	Histórico. Mayor longitud de periodo que Cuadrados Medios.
Blum Blum Shub	$X_{i+1} = (X_i)^2 \bmod (p \cdot q)$	Divide a $\lambda(\lambda(M))$	Criptográficamente Seguro (CSPRNG) . Generación de claves y cifrado.

4. Motor de Detección de Ciclos y Estadísticas

En core/generators.py (función calculate_stats()) se implementó un motor analítico que evalúa en tiempo de ejecución:

- **Detección de Ciclos por Memoria de Estados (Hash Map O(1)):** Durante la iteración, cada estado generado X_i (o el par (X_{i-1}, X_i) en Productos Medios) se indexa en una tabla hash. En el instante exacto en que un estado se repite, se registra el inicio del bucle, se marca la fila en la tabla visual y se calcula el periodo exacto $\text{Periodo} = i_{\text{actual}} - i_{\text{previo}}$.
- **Análisis de Independencia Estocástica (Dispersión de Retardo Lag-1):** Se construyen los pares ordenados (R_i, R_{i+1}) en el plano unitario $[0, 1) \times [0, 1)$. Permite detectar visual y cuantitativamente la presencia de hiperplanos o patrones deterministas.
- **Coefficiente de Autocorrelación Lag-1 ($\rho_{\text{Lag-1}}$):** Se computa el estadístico formal $\rho_{\text{Lag-1}} = \text{Cov}(R_i, R_{i+1}) / \text{Var}(R_i)$. Para una secuencia verdaderamente independiente, $\rho_{\text{Lag-1}} \approx 0$.